

Project INF224 - Multimedia Set-Top Box

Author : BALI Samy

This project was developed on macOS; therefore, the system calls used are specific to macOS. In particular, the “play” functions of the multimedia objects must be modified to be compatible with other operating systems (e.g., using xdg-open on Linux or start on Windows).

To compile the C++ part, you must use the two provided Makefiles. Then, the server must be started before starting the Java GUI.

All steps of both TPs (C++ and Java) have been completed. Only the optional parts of a few steps have been skipped.

The following are the answers to the questions asked during the TPs:

Step 4:

- ***What do we call this type of method and how should it be declared?***
It is called an abstract method. It must be declared ;with = 0 in the base class. By doing this we force the children classes to implement the method
- ***If you followed the previous steps, it is no longer possible to instantiate objects of the base class. Why?***
Because the Multimedia class became abstract. Therefore we cannot instantiate objects of the base class and derived classes need to implement the abstract methods.

Step 5:

- ***What is the characteristic property of object-oriented programming that allows this?***
It is polymorphism. It allows objects of different types to be treated the same way via a pointer to their base class.
- ***What is specifically required to do this in C++?***
In C++, two conditions are needed for polymorphism to work: first we use methods declared as virtual in the base class and we manipulate objects via pointers of the base class.
- ***What is the type of the array elements: should the array contain objects or pointers to these objects? Why? Compare with Java.***
The array must contain pointers. If we used an array of objects, we would encounter the object slicing problem: only the part inherited from the base class of the object would be copied. In Java, this issue does not arise because collections always store references to objects, never the objects themselves.

Step 6:

- ***What should the accessor for the duration array return?***

It must return a constant pointer or a copy of the array to prevent the user from directly modifying the internal array content and breaking encapsulation.

- ***How to handle deep copy (copy constructor and assignment operator)?***

Since the Film class contains a pointer, we redefine the operator “=”. If we rely on the default shallow copy, two Film objects would point to the same memory area for chapters. If one is destroyed, the other would hold an invalid pointer. This is why we need to redefine the “=” operator.

Step 7:

- ***Among the previously written classes, which ones need modification to prevent memory leaks?***

The Film class. Its destructor must be implemented to release the memory dynamically allocated for the chapter array. We call delete[] to correctly delete the chapters array.

Step 8:

- ***The list of objects must actually be a list of object pointers. Why? Compare with Java.***

A std::list<Multimedia> would store copies of the objects that only have the parent part of the class. We need a std::list<Multimedia*> in order to manage the classes with their methods. In Java, references are implicitly stored. Moreover, in C++, an object can belong to multiple groups. Storing pointers ensures the object is only defined once in the memory.

Step 10:

- ***How can we prevent object creation using new (outside the Manager)?***

We can declare the constructors of the Photo, Video, and Film classes as protected and declare the Manager class as a friend within these classes. Therefore, only the Manager is allowed to instantiate these objects.

I added two methods for group and multimedia removal. I decided that the remove method for a group should not remove the elements that were inside because when we want to for example remove a folder, we do not want everything to be removed.

Step 1 Java:

- ***Run your program and resize the window. What do you observe?***

If we add a lot of text, it disappears or goes out of bounds without the ability to scroll. Therefore, we need to encapsulate the JTextArea inside a JScrollPane in order to enable automatic scrollbars.

The documentation has been generated with Doxygen. In order to be able to navigate through it, one can open the index.html in the doc folder. We also have a diagram in order to visualize the inheritance of the classes used.